

Loan Prediction Dataset Analysis Report

This project analyzes the Loan Prediction dataset to identify key factors influencing loan approval decisions. The dataset contains attributes such as applicant income, education level, employment status, loan amount, and property area. The objective of this analysis is to perform data preprocessing, explore patterns using visualization, and understand relationships between variables. Techniques such as distribution analysis, correlation analysis, encoding, and scaling are applied to prepare the dataset for machine learning models.

About the Dataset

The Loan Prediction dataset contains information about loan applicants and is used to analyze the factors that influence loan approval decisions.

Categorical Features:

Gender, Married, Dependents, Education, Self_Employed, Property_Area

Numerical Features:

ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History

Target Variable:

Loan_Status (Y = Approved, N = Not Approved)

Type *Markdown* and LaTeX: α^2

First, I imported the required libraries like pandas, numpy, matplotlib, and seaborn. Then I loaded the dataset using pandas read_csv function and performed basic inspection like checking shape, columns, and data types

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: df = pd.read_csv("loanprediction.csv")
```

In [3]: `df.head()`

Out[3]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coappl
0	LP001002	Male	No	0	Graduate	No	5849	
1	LP001003	Male	Yes	1	Graduate	No	4583	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	
4	LP001008	Male	No	0	Graduate	No	6000	

In [4]: `print("Rows and Columns:", df.shape)`

Rows and Columns: (614, 13)

The dataset contains 614 rows and 13 columns, representing 614 records and 13 features in the dataset

In [5]: `print(df.columns)`

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
      'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
      'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Loan_ID               614 non-null   object  
1   Gender                 601 non-null   object  
2   Married                611 non-null   object  
3   Dependents             599 non-null   object  
4   Education              614 non-null   object  
5   Self_Employed          582 non-null   object  
6   ApplicantIncome        614 non-null   int64   
7   CoapplicantIncome      614 non-null   float64  
8   LoanAmount             592 non-null   float64  
9   Loan_Amount_Term       600 non-null   float64  
10  Credit_History         564 non-null   float64  
11  Property_Area          614 non-null   object  
12  Loan_Status            614 non-null   object  
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

In [7]: `df.describe()`

Out[7]:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000
max	81000.000000	41667.000000	700.000000	480.000000	1.000000

In [8]: `df.dtypes`

Out[8]:

Loan_ID	object
Gender	object
Married	object
Dependents	object
Education	object
Self_Employed	object
ApplicantIncome	int64
CoapplicantIncome	float64
LoanAmount	float64
Loan_Amount_Term	float64
Credit_History	float64
Property_Area	object
Loan_Status	object
dtype:	object

In [9]: `df.duplicated().sum()`

Out[9]: 0

In [10]: `df.nunique()`

Out[10]:

Loan_ID	614
Gender	2
Married	2
Dependents	4
Education	2
Self_Employed	2
ApplicantIncome	505
CoapplicantIncome	287
LoanAmount	203
Loan_Amount_Term	10
Credit_History	2
Property_Area	3
Loan_Status	2
dtype:	int64

```
In [11]: df['Education'].unique()
```

```
Out[11]: array(['Graduate', 'Not Graduate'], dtype=object)
```

```
In [ ]:
```

Data Cleaning (Missing Values, Duplicates, Renaming)

```
In [12]: df.isnull().sum()
```

```
Out[12]: Loan_ID          0
Gender          13
Married         3
Dependents      15
Education        0
Self_Employed   32
ApplicantIncome  0
CoapplicantIncome  0
LoanAmount      22
Loan_Amount_Term 14
Credit_History  50
Property_Area    0
Loan_Status      0
dtype: int64
```

The dataset contains missing values in several columns. The highest missing values are in Credit_History (50), followed by Self_Employed (32) and LoanAmount (22). Other columns like Gender, Dependents, and Loan_Amount_Term have fewer missing values, while some columns such as Loan_ID, Education, ApplicantIncome, CoapplicantIncome, Property_Area, and Loan_Status have no missing values.

```
In [13]: #Remove unnecessary column (Loan_ID)
```

```
In [14]: df = df.drop(columns=['Loan_ID'])
```

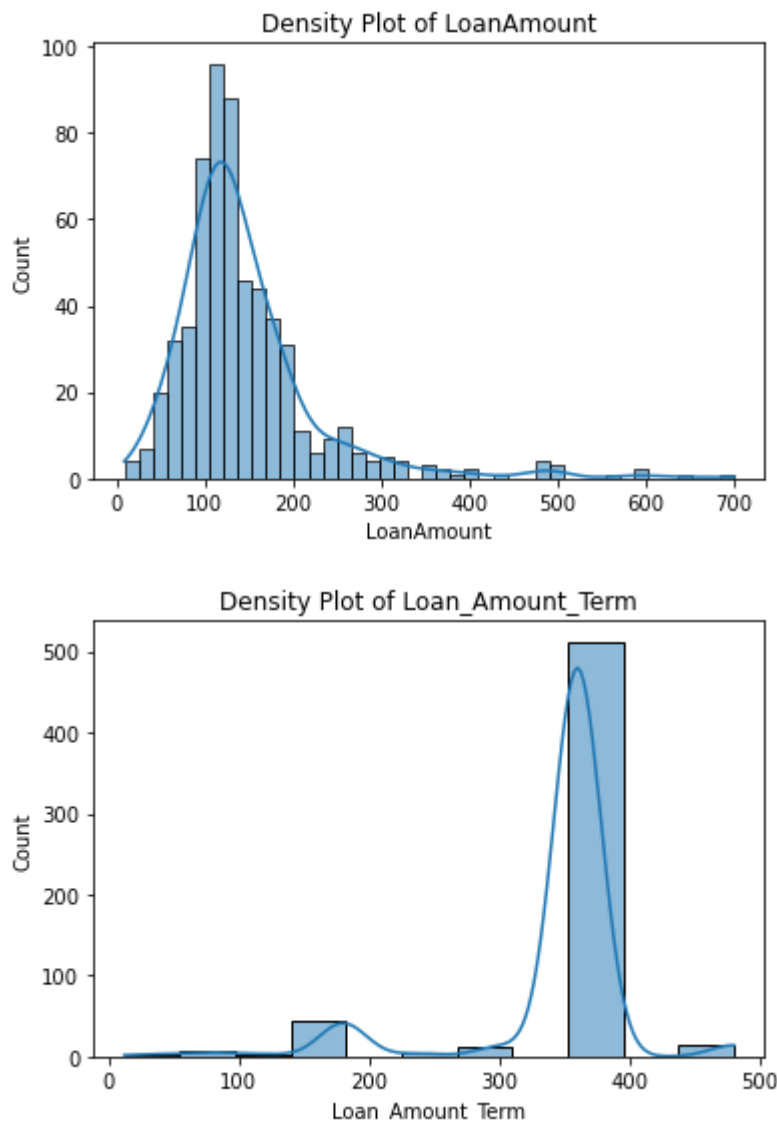
Missing values were handled using mode for categorical variables and median for numerical variables, as median handles skewed data and mode maintains category consistency.

```
In [15]: # List of categorical columns
cat_cols = ['Gender', 'Married', 'Dependents', 'Self_Employed']

# Fill missing values with mode using loop
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
In [16]: num_cols = ['LoanAmount', 'Loan_Amount_Term']
```

```
In [17]: for col in num_cols:
plt.figure(figsize=(6,4))
sns.histplot(df[col], fill=True,kde=True)
plt.title(f'Density Plot of {col}')
plt.show()
```



Loan Amount -> -Distribution is right-skewed (positively skewed)

-Long tail extending towards higher values (300–700)

Loan_Amount_Term-> -Most values concentrated around 360

-Looks almost like a peak (not perfectly normal)

-Slight irregular distribution but not heavily skewed

```
In [18]: # Fill missing values using median

for col in ['LoanAmount', 'Loan_Amount_Term']:
    df[col] = df[col].fillna(df[col].median())
```

```
In [19]: df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode())
```

```
In [20]: df.isnull().sum()
```

```
Out[20]: Gender                0
Married                0
Dependents              0
Education              0
Self_Employed          0
ApplicantIncome        0
CoapplicantIncome      0
LoanAmount             0
Loan_Amount_Term       0
Credit_History         0
Property_Area          0
Loan_Status            0
dtype: int64
```

```
In [21]: #Rename the Columns
```

```
In [22]: df.rename(columns={
    'ApplicantIncome': 'Applicant_Income',
    'CoapplicantIncome': 'Coapplicant_Income',
    'LoanAmount': 'Loan_Amount',
    'Loan_Amount_Term': 'Loan_Term',
}, inplace=True)

# Check updated columns
print(df.columns)
```

```
Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
      'Applicant_Income', 'Coapplicant_Income', 'Loan_Amount', 'Loan_Term',
      'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

```
In [23]: cat_cols = df.select_dtypes(include='object').columns

for col in cat_cols:
    print(f"\n{col} Value Counts:\n")
    print(df[col].value_counts())
```

Gender Value Counts:

```
Male      502
Female    112
Name: Gender, dtype: int64
```

Married Value Counts:

```
Yes      401
No       213
Name: Married, dtype: int64
```

Dependents Value Counts:

```
0      360
1      102
2      101
3+      51
Name: Dependents, dtype: int64
```

Education Value Counts:

```
Graduate      480
Not Graduate   134
Name: Education, dtype: int64
```

Self_Employed Value Counts:

```
No      532
Yes      82
Name: Self_Employed, dtype: int64
```

Property_Area Value Counts:

```
Semiurban    233
Urban        202
Rural        179
Name: Property_Area, dtype: int64
```

Loan_Status Value Counts:

```
Y      422
N      192
Name: Loan_Status, dtype: int64
```

In []:

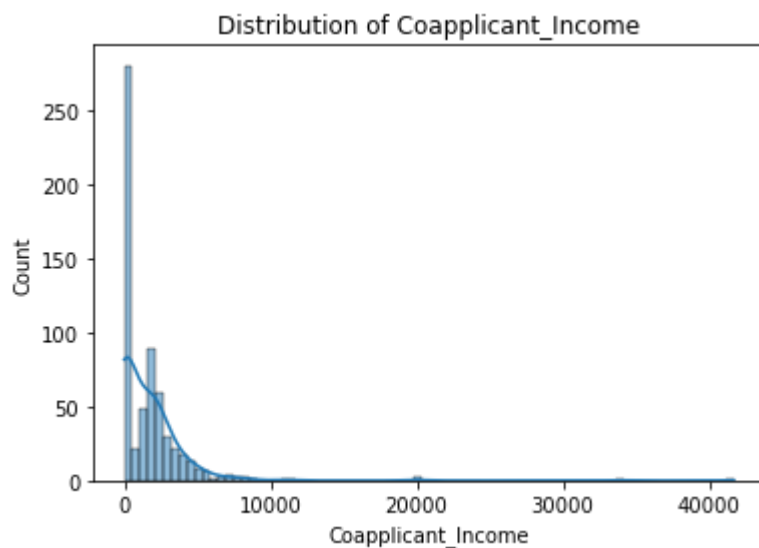
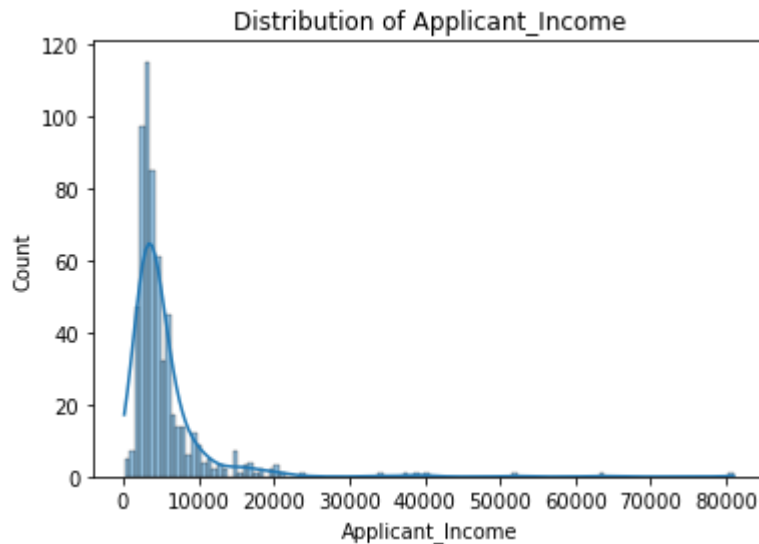
Univariate Analysis (Distributions, Boxplots, Outliers)

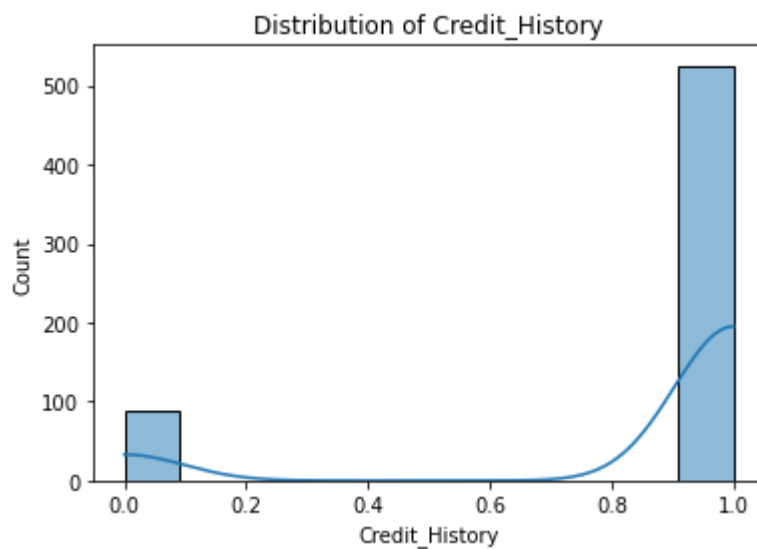
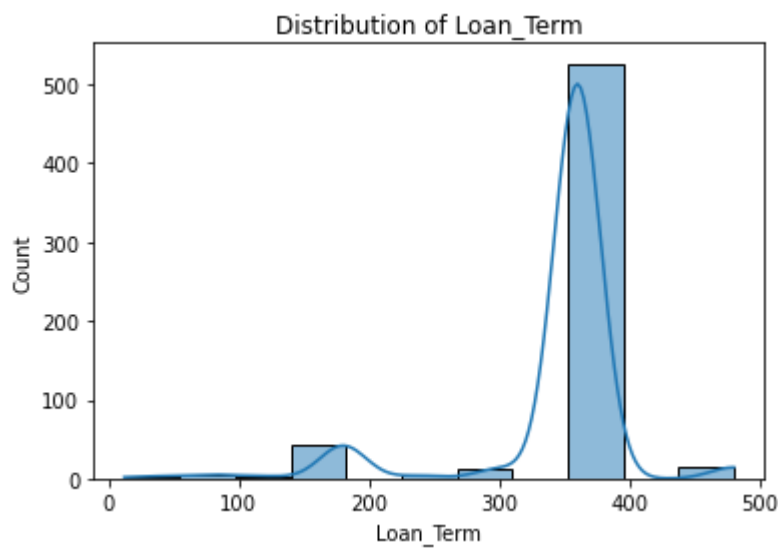
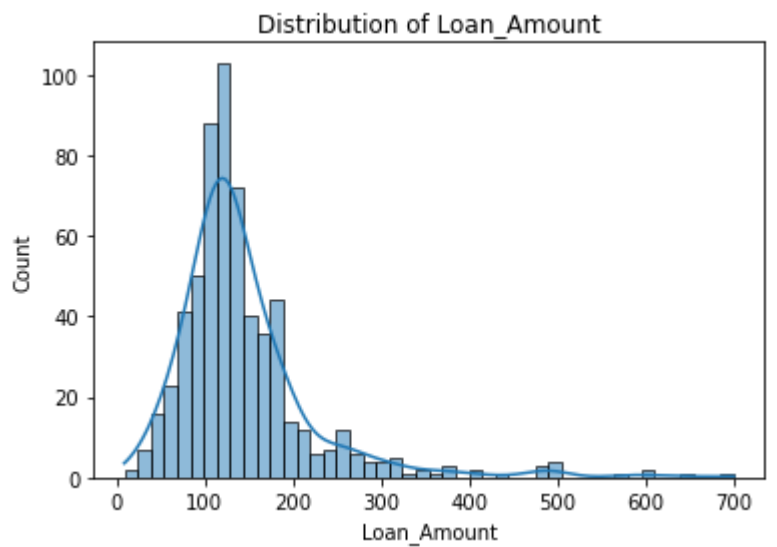
Distribution Plot

```
In [24]: import seaborn as sns
import matplotlib.pyplot as plt

num_cols = df.select_dtypes(include=['int64', 'float64']).columns

for col in num_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.show()
```





ApplicantIncome → Highly right-skewed (few very high incomes)

CoapplicantIncome → Many zero values, skewed

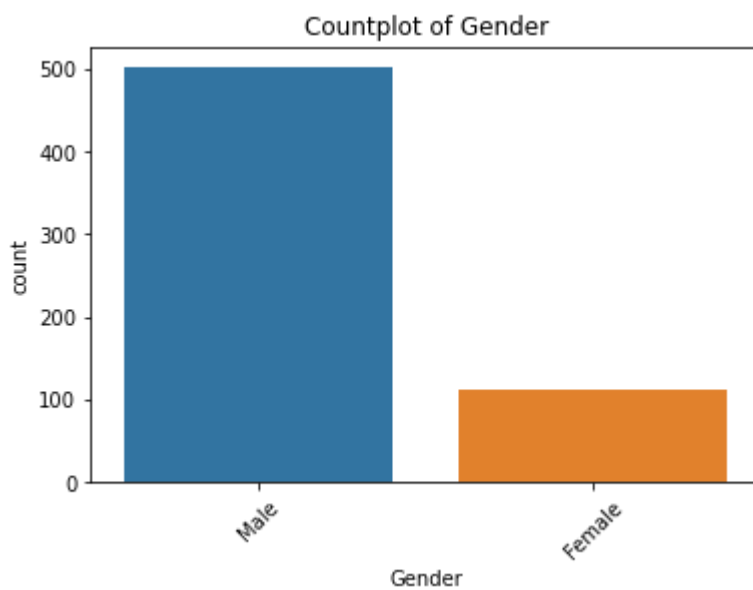
LoanAmount → Right-skewed with extreme values

Loan_Amount_Term → Most values concentrated at 360

In []:

Countplot

```
In [25]: cat_cols = ['Gender', 'Married', 'Dependents', 'Education',  
                    'Self_Employed', 'Property_Area',  
                    'Loan_Status']  
  
for col in cat_cols:  
    plt.figure(figsize=(6,4))  
    sns.countplot(x=df[col])  
    plt.title(f'Countplot of {col}')  
    plt.xticks(rotation=45)  
    plt.show()
```



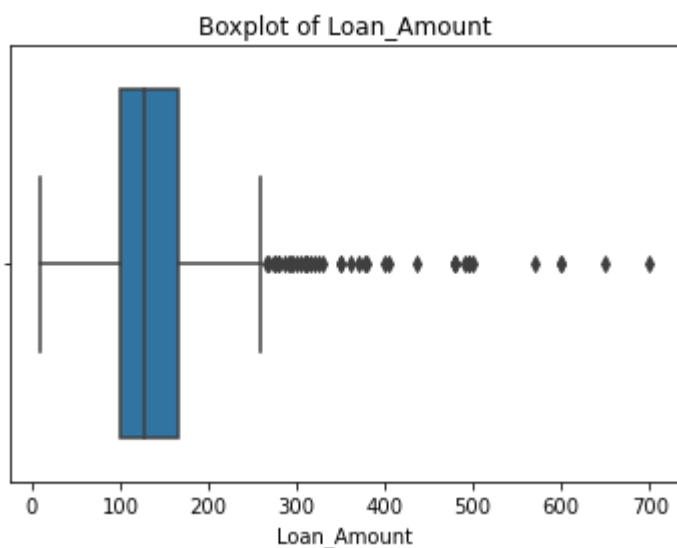
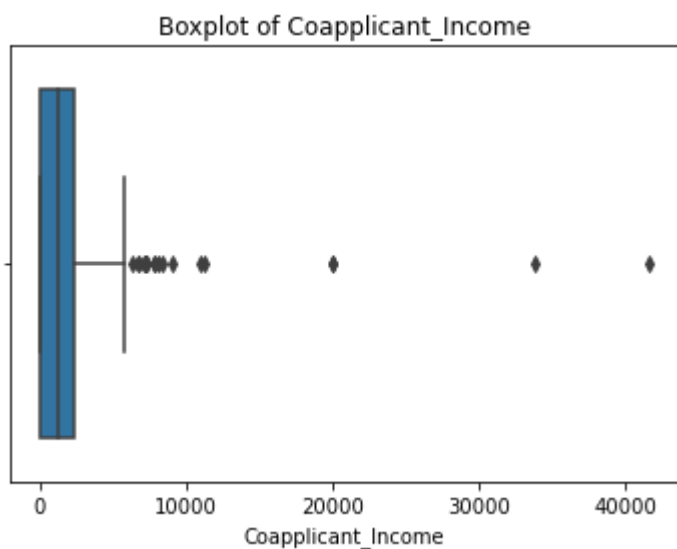
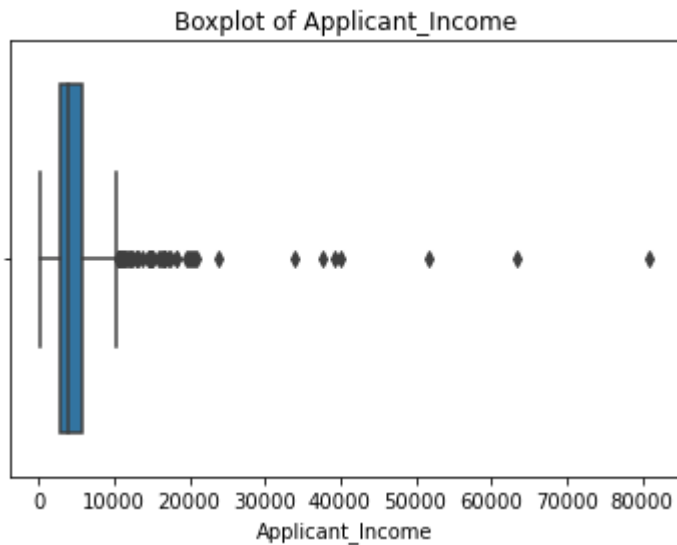
Countplot of Married

Box Plot (Outliers Detection)

In [26]:

```
outlier_cols = ['Applicant_Income', 'Coapplicant_Income', 'Loan_Amount']

for col in outlier_cols:
    plt.figure(figsize=(6,4))
    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')
    plt.show()
```



```
In [27]: for col in outlier_cols:
          Q1 = df[col].quantile(0.25)
          Q3 = df[col].quantile(0.75)
          IQR = Q3 - Q1

          lower = Q1 - 1.5 * IQR
          upper = Q3 + 1.5 * IQR

          outliers = df[(df[col] < lower) | (df[col] > upper)]

          print(f"{col}: {len(outliers)} outliers")
```

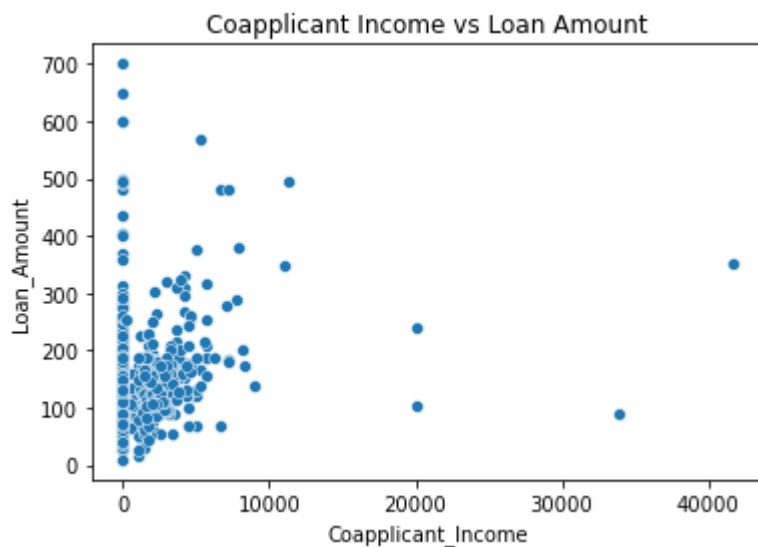
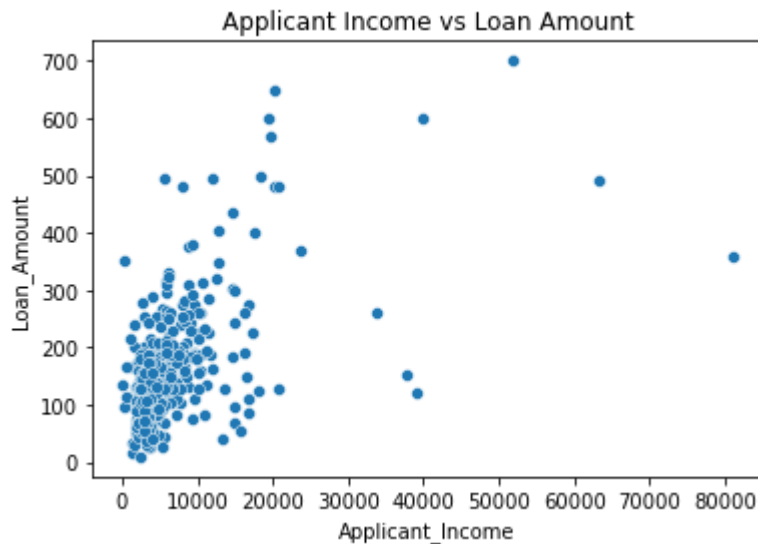
```
Applicant_Income: 50 outliers
Coapplicant_Income: 18 outliers
Loan_Amount: 41 outliers
```

In []:

Bivariate analysis (scatterplots, correlations, groupby analysis)

```
In [28]: # Applicant Income vs Loan Amount
sns.scatterplot(x=df['Applicant_Income'], y=df['Loan_Amount'])
plt.title('Applicant Income vs Loan Amount')
plt.show()

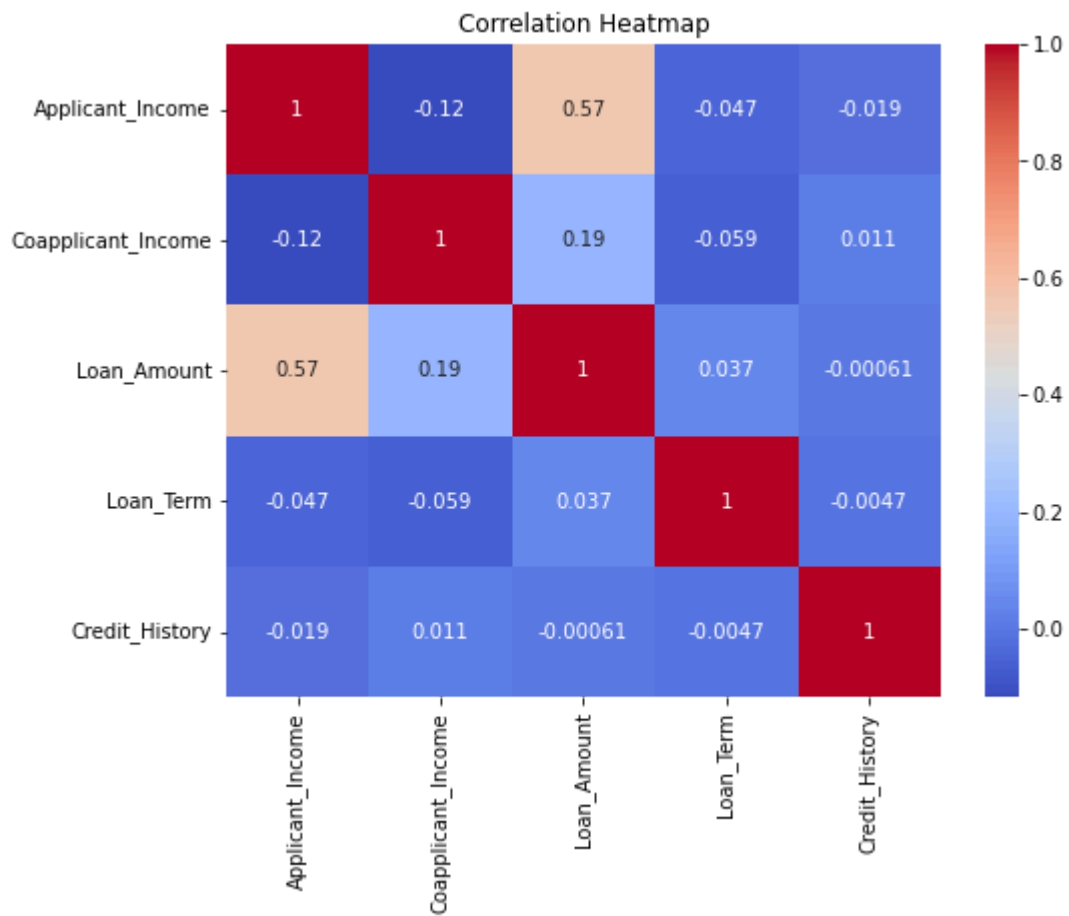
# Coapplicant Income vs Loan Amount
sns.scatterplot(x=df['Coapplicant_Income'], y=df['Loan_Amount'])
plt.title('Coapplicant Income vs Loan Amount')
plt.show()
```



Applicant Income vs Loan Amount: There is a weak positive relationship → higher income sometimes leads to higher loan, but not always (points are scattered). Coapplicant Income vs Loan Amount: Relationship is very weak → coapplicant income has less impact on loan amount.

```
In [29]: corr = df.corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



Applicant Income & Loan Amount : Moderate positive correlation → higher income → higher loan

Coapplicant Income & Loan Amount : Weak positive relationship

Loan Term & others: Very weak / no correlation

Credit History: Almost no linear relation with income or loan amount

```
In [30]: # Average Loan Amount by Property Area
print(df.groupby('Property_Area')['Loan_Amount'].mean())

# Loan approval count by Gender
print(df.groupby('Gender')['Loan_Status'].value_counts())

# Loan approval by Education
print(df.groupby('Education')['Loan_Status'].value_counts())
```

```
Property_Area
Rural      151.446927
Semiurban  145.128755
Urban      141.425743
Name: Loan_Amount, dtype: float64
Gender  Loan_Status
Female  Y           75
        N           37
Male    Y          347
        N          155
Name: Loan_Status, dtype: int64
Education  Loan_Status
Graduate   Y           340
           N           140
Not Graduate Y           82
           N           52
Name: Loan_Status, dtype: int64
```

```
In [31]: sns.countplot(x='Education', hue='Loan_Status', data=df)
plt.title('Education vs Loan Status')
plt.show()

sns.countplot(x='Property_Area', hue='Loan_Status', data=df)
plt.title('Property Area vs Loan Status')
plt.show()
```



Education vs Loan Status: Graduates have more loan approvals (Y) compared to non-graduates → education increases approval chances.

Property Area vs Loan Status: Semiurban areas have the highest loan approvals, while rural areas have relatively fewer approvals.

In []:

Encoding and Scaling

Label Encoding was applied to convert categorical variables into a numerical format: Gender, Married, Dependents, Education, Self_Employed, Property_Area, Loan_Status

```
In [32]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
cols_to_encode = ['Gender', 'Married', 'Dependents', 'Education',
                  'Self_Employed', 'Property_Area', 'Loan_Status']

for col in cols_to_encode:
    df[col] = le.fit_transform(df[col])
```

```
In [33]: df
```

```
Out[33]:
```

	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_In
0	1	0	0	0	0	5849	
1	1	1	1	0	0	4583	1
2	1	1	0	0	1	3000	
3	1	1	0	1	0	2583	2
4	1	0	0	0	0	6000	
...	...	...	...	...	...	...	
609	0	0	0	0	0	2900	
610	1	1	3	0	0	4106	
611	1	1	1	0	0	8072	
612	1	1	2	0	0	7583	
613	0	0	0	0	1	4583	

614 rows × 12 columns



```
In [ ]:
```

Scaling

Min-Max Scaling was applied to normalize numerical values to the range [0, 1], ensuring equal contribution from all features.

```
In [34]: x = df.drop(columns=['Loan_Status'])
y = df['Loan_Status']
```

In [35]: y

```
Out[35]: 0      1
          1      0
          2      1
          3      1
          4      1
          ..
        609      1
        610      1
        611      1
        612      1
        613      0
Name: Loan_Status, Length: 614, dtype: int32
```

```
In [36]: from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
```

In [37]: x

```
Out[37]:
```

	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_In
0	1	0	0	0	0	5849	
1	1	1	1	0	0	4583	1
2	1	1	0	0	1	3000	
3	1	1	0	1	0	2583	2
4	1	0	0	0	0	6000	
...	...	...	...	...	...	...	
609	0	0	0	0	0	2900	
610	1	1	3	0	0	4106	
611	1	1	1	0	0	8072	
612	1	1	2	0	0	7583	
613	0	0	0	0	1	4583	

614 rows × 11 columns



```
In [38]: x1=scaler.fit_transform(x)
x1
```

```
Out[38]: array([[1.          , 0.          , 0.          , ..., 0.74358974, 1.          ,
                1.          ],
                [1.          , 1.          , 0.33333333, ..., 0.74358974, 1.          ,
                0.          ],
                [1.          , 1.          , 0.          , ..., 0.74358974, 1.          ,
                1.          ],
                ...,
                [1.          , 1.          , 0.33333333, ..., 0.74358974, 1.          ,
                1.          ],
                [1.          , 1.          , 0.66666667, ..., 0.74358974, 1.          ,
                1.          ],
                [0.          , 0.          , 0.          , ..., 0.74358974, 0.          ,
                0.5         ]])
```

```
In [39]: x2=pd.DataFrame(x1,columns=[ 'Gender', 'Married', 'Dependents', 'Education',
                                       'Self_Employed', 'Applicant_Income', 'Coapplicant_Income',
                                       'Loan_Amount', 'Loan_Term', 'Credit_History', 'Property_Area'])
```

```
In [40]: x2
```

```
Out[40]:
```

	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_In
0	1.0	0.0	0.000000	0.0	0.0	0.070489	0.00
1	1.0	1.0	0.333333	0.0	0.0	0.054830	0.00
2	1.0	1.0	0.000000	0.0	1.0	0.035250	0.00
3	1.0	1.0	0.000000	1.0	0.0	0.030093	0.00
4	1.0	0.0	0.000000	0.0	0.0	0.072356	0.00
...	...	...	...	...	...	...	...
609	0.0	0.0	0.000000	0.0	0.0	0.034014	0.00
610	1.0	1.0	1.000000	0.0	0.0	0.048930	0.00
611	1.0	1.0	0.333333	0.0	0.0	0.097984	0.00
612	1.0	1.0	0.666667	0.0	0.0	0.091936	0.00
613	0.0	0.0	0.000000	0.0	1.0	0.054830	0.00

614 rows × 11 columns



```
In [ ]:
```

```
In [41]: x2=pd.concat([x2,y],axis=1)
x2
```

```
Out[41]:
```

	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_In
0	1.0	0.0	0.000000	0.0	0.0	0.070489	0.00
1	1.0	1.0	0.333333	0.0	0.0	0.054830	0.00
2	1.0	1.0	0.000000	0.0	1.0	0.035250	0.00
3	1.0	1.0	0.000000	1.0	0.0	0.030093	0.00
4	1.0	0.0	0.000000	0.0	0.0	0.072356	0.00
...	...	...	...	...	...	...	...
609	0.0	0.0	0.000000	0.0	0.0	0.034014	0.00
610	1.0	1.0	1.000000	0.0	0.0	0.048930	0.00
611	1.0	1.0	0.333333	0.0	0.0	0.097984	0.00
612	1.0	1.0	0.666667	0.0	0.0	0.091936	0.00
613	0.0	0.0	0.000000	0.0	1.0	0.054830	0.00

614 rows × 12 columns

```
In [42]: x2.describe()
```

```
Out[42]:
```

	Gender	Married	Dependents	Education	Self_Employed	Applicant_Income	Coapplicant_In
count	614.000000	614.000000	614.000000	614.000000	614.000000	614.000000	614.000000
mean	0.817590	0.653094	0.248100	0.218241	0.133550	0.064978	0.064978
std	0.386497	0.476373	0.336541	0.413389	0.340446	0.075560	0.075560
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	0.000000	0.000000	0.000000	0.033735	0.033735
50%	1.000000	1.000000	0.000000	0.000000	0.000000	0.045300	0.045300
75%	1.000000	1.000000	0.333333	0.000000	0.000000	0.069821	0.069821
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

```
In [ ]:
```

Conclusion

In this project, I worked on the Loan Prediction dataset by cleaning and preparing the data using preprocessing techniques. I handled missing values using median and mode, and converted categorical data into a suitable format for analysis. Through univariate and bivariate analysis, I understood the data distribution and relationships between different features. I found that factors like income, loan amount, and credit history play an important role in loan approval. Finally, the dataset was well-prepared for building machine learning models to improve prediction accuracy.

```
In [43]: df=x2
```

```
In [44]: x=df.drop(['Loan_Status'],axis=1)
y=df['Loan_Status']
```

```
In [55]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=
```

```
In [56]: from sklearn.linear_model import LogisticRegression
logreg=LogisticRegression()
```

```
In [57]: model=logreg.fit(x_train,y_train)
```

```
In [58]: y_pred=model.predict(x_test)
```

```
In [59]: from sklearn.metrics import accuracy_score
print('accuracy',accuracy_score(y_test,y_pred))io
```

```
accuracy 0.7886178861788617
```

```
In [60]: from sklearn.svm import SVC
svc=SVC(kernel='rbf')
```

```
In [61]: model=svc.fit(x_train,y_train)
```

```
In [62]: y_pred=model.predict(x_test)
```

```
In [63]: from sklearn.metrics import accuracy_score
print('accuracy',accuracy_score(y_test,y_pred))
```

```
accuracy 0.7886178861788617
```

```
In [67]: from sklearn.tree import DecisionTreeClassifier
dtc=DecisionTreeClassifier(random_state=42)
```

```
In [ ]: model=dtc.fit(x_train,y_train)
```